

Jetpack Compose第三章第二节教学讲义（输入框实战全解析）

各位同学，输入框是APP中最常用的组件之一——登录页的账号密码、聊天页的消息输入、设置页的信息修改，都离不开它。但看似简单的输入框，藏着不少“细节坑”：比如输入框下方要加提示图标、软键盘挡住输入内容、切换编辑/查看模式、页面重建后保留输入内容等。这节课咱们通过6个实战案例，逐一攻克这些问题，每个案例都配完整代码，保证你学完就能直接用在项目里！

案例一：输入框下方添加图标（提示与反馈）

场景：注册页面的密码输入框，下方需要添加“密码强度提示图标”（如弱/中/强），或“格式错误提示图标”——这是提升用户体验的常用设计。

核心思路

用 `Column` 垂直排列“输入框”和“图标+提示文本”，通过输入内容的长度或格式，动态控制图标的颜色和提示文本。

完整代码实现

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.Row
3  import androidx.compose.foundation.layout.padding
4  import androidx.compose.foundation.layout.size
5  import androidx.compose.material.icons.Icons
6  import androidx.compose.material.icons.filled.Info
7  import androidx.compose.material.icons.filled.CheckCircle
8  import androidx.compose.material3.Icon
9  import androidx.compose.material3.OutlinedTextField
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableStateOf
14 import androidx.compose.runtime.remember
15 import androidx.compose.runtime.setValue
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.unit.dp
```

```

20 import androidx.compose.ui.tooling.preview.Preview
21
22 @Composable
23 fun PasswordFieldWithIcon() {
24     // 状态：密码输入内容
25     var password by remember { mutableStateOf("") }
26
27     // 根据密码长度判断强度（实际项目可加正则判断格式）
28     val passwordStrength = when {
29         password.length < 6 -> "弱"
30         password.length < 10 -> "中"
31         else -> "强"
32     }
33
34     // 根据强度确定图标和颜色
35     val (icon, color) = when (passwordStrength) {
36         "弱" -> Pair(Icons.Default.Info, Color.Red)
37         "中" -> Pair(Icons.Default.Info, Color.Yellow)
38         "强" -> Pair(Icons.Default.CheckCircle, Color.Green)
39         else -> Pair(Icons.Default.Info, Color.Gray)
40     }
41
42     Column(modifier = Modifier.padding(16.dp)) {
43         // 密码输入框
44         OutlinedTextField(
45             value = password,
46             onValueChange = { password = it },
47             label = { Text("请输入密码") },
48             placeholder = { Text("6-16位字母+数字") }
49         )
50
51         // 输入框下方的图标+提示
52         Row(
53             modifier = Modifier.padding(top = 4.dp),
54             verticalAlignment = Alignment.CenterVertically
55         ) {
56             Icon(
57                 imageVector = icon,
58                 contentDescription = "密码强度提示",
59                 modifier = Modifier.size(16.dp),
60                 tint = color
61             )
62             Text(
63                 text = "密码强度: $passwordStrength",
64                 modifier = Modifier.padding(start = 4.dp),
65                 color = color,
66                 fontSize = 12.sp

```

```

67         )
68     }
69 }
70 }
71
72 @Preview(showBackground = true)
73 @Composable
74 fun PasswordFieldPreview() {
75     PasswordFieldWithIcon()
76 }

```

关键知识点

- 用 `Column` 实现垂直布局，确保图标在输入框正下方；
- 通过输入内容动态切换UI（图标、颜色、文本），这是“状态驱动UI”的典型应用；
- 实际项目中，可替换密码强度判断逻辑（如用正则匹配是否包含字母、数字、特殊符号）。

案例二：软键盘适配（避免输入框被挡住）

场景：页面底部的输入框（如聊天APP的消息输入框），点击后软键盘弹出，会挡住输入框——这是新手最常遇到的软键盘问题。

核心思路

通过两个步骤解决：① 在AndroidManifest.xml中给Activity添加“软键盘适配”配置；② 用 `Modifier.imePadding()` 给输入框添加“软键盘高度的内边距”。

步骤1：配置AndroidManifest.xml

找到对应页面的Activity，添加 `android:windowSoftInputMode="adjustResize"`，让页面在软键盘弹出时自动调整大小：

Code block

```

1  <activity
2      android:name=".ui.ChatActivity"
3      android:windowSoftInputMode="adjustResize" />

```

步骤2：Compose代码实现软键盘适配

Code block

```

1  import androidx.compose.foundation.layout.Box

```

```

2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.foundation.layout.imePadding
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.material3.OutlinedTextField
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.runtime.getValue
9  import androidx.compose.runtime.mutableStateOf
10 import androidx.compose.runtime.remember
11 import androidx.compose.runtime.setValue
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.tooling.preview.Preview
15
16 @Composable
17 fun ChatInputField() {
18     // 状态：聊天输入内容
19     var message by remember { mutableStateOf("") }
20
21     // 用Box让输入框固定在底部
22     Box(
23         modifier = Modifier
24             .fillMaxSize()
25             .padding(16.dp)
26     ) {
27         OutlinedTextField(
28             value = message,
29             onValueChange = { message = it },
30             label = { Text("输入消息") },
31             modifier = Modifier
32                 .align(Alignment.BottomCenter) // 固定在底部
33                 .fillMaxWidth()
34                 .imePadding() // 关键：添加软键盘高度的内边距
35         )
36     }
37 }
38
39 @Preview(showBackground = true)
40 @Composable
41 fun ChatInputPreview() {
42     ChatInputField()
43 }

```

关键知识点

`Modifier.imePadding()` 是Compose提供的“软键盘适配专属修饰符”，它会自动识别软键盘的高度，给组件添加底部内边距——软键盘弹出时生效，收起时自动消失，无需手动计算高度。

案例三：自定义输入框背景（摆脱默认样式）

场景：设计稿要求输入框是“圆角、浅蓝色背景、无外边框”，但Compose默认的OutlinedTextField是“带边框的样式”——需要自定义输入框背景。

核心思路

用 `Box` 作为“背景容器”，先绘制自定义背景（圆角、颜色），再把 `BasicTextField`（无默认样式的输入框）放在背景上方——BasicTextField比OutlinedTextField更灵活，适合完全自定义样式。

完整代码实现

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.border
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.foundation.shape.RoundedCornerShape
6  import androidx.compose.material3.BasicTextField
7  import androidx.compose.material3.Text
8  import androidx.compose.runtime.Composable
9  import androidx.compose.runtime.getValue
10 import androidx.compose.runtime.mutableStateOf
11 import androidx.compose.runtime.remember
12 import androidx.compose.runtime.setValue
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.text.input.TextFieldValue
16 import androidx.compose.ui.unit.dp
17 import androidx.compose.ui.tooling.preview.Preview
18
19 @Composable
20 fun CustomBackgroundTextField() {
21     // 状态：输入内容（用TextFieldValue可获取光标位置等更多信息）
22     var textState by remember { mutableStateOf(TextFieldValue("")) }
23
24     // 自定义提示文本（BasicTextField没有默认label，需手动实现）
25     val hintText = "请输入搜索内容"
26
27     Box(
28         modifier = Modifier
29             .fillMaxWidth()
```

```

30         .padding(16.dp)
31     ) {
32         // 步骤1: 绘制自定义背景 (圆角、浅蓝色、细边框)
33         Box(
34             modifier = Modifier
35                 .fillMaxWidth()
36                 .background(
37                     color = Color(0xFFF5F8FF), // 浅蓝色背景
38                     shape = RoundedCornerShape(20.dp) // 圆角
39                 )
40                 .border(
41                     width = 1.dp,
42                     color = Color(0xFFE1E7F5), // 边框颜色
43                     shape = RoundedCornerShape(20.dp)
44                 )
45                 .padding(horizontal = 16.dp, vertical = 12.dp)
46         )
47
48         // 步骤2: 在背景上放置输入框 (BasicTextField)
49         BasicTextField(
50             value = textState,
51             onValueChange = { textState = it },
52             modifier = Modifier.fillMaxWidth(),
53             // 输入框为空时显示提示文本
54             decorationBox = { innerTextField ->
55                 if (textState.text.isEmpty()) {
56                     Text(
57                         text = hintText,
58                         color = Color.Gray,
59                         fontSize = 14.sp
60                     )
61                 }
62                 innerTextField() // 必须调用, 显示输入的文本
63             }
64         )
65     }
66 }
67
68 @Preview(showBackground = true)
69 @Composable
70 fun CustomTextFieldPreview() {
71     CustomBackgroundTextField()
72 }

```

关键知识点

- `BasicTextField` 是“无样式输入框”，适合完全自定义；`OutlinedTextField` 和 `FilledTextField` 是带默认样式的，适合快速开发；
- `decorationBox` 参数用于添加提示文本、图标等装饰内容，是自定义输入框的核心API；
- 背景色和边框色建议用设计稿的十六进制色值（如0xFF5F8FF），保证样式一致。

案例四：输入框切换编辑/查看模式（如个人资料页）

场景：个人资料页的“昵称”展示——默认是“查看模式”（文本显示，无输入框），点击“编辑”按钮后切换为“编辑模式”（显示输入框，可修改内容）。

核心思路

用一个布尔状态 `isEditing` 控制模式：① 查看模式显示 `Text` 组件；② 编辑模式显示 `OutlinedTextField`，并自动获取焦点（方便用户直接输入）。

完整代码实现

Code block

```
1  import androidx.compose.foundation.layout.Row
2  import androidx.compose.foundation.layout.fillMaxWidth
3  import androidx.compose.foundation.layout.spaceBetween
4  import androidx.compose.material3.Button
5  import androidx.compose.material3.OutlinedTextField
6  import androidx.compose.material3.Text
7  import androidx.compose.runtime.Composable
8  import androidx.compose.runtime.getValue
9  import androidx.compose.runtime.mutableStateOf
10 import androidx.compose.runtime.remember
11 import androidx.compose.runtime.setValue
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.focus.FocusRequester
15 import androidx.compose.ui.focus.focusRequester
16 import androidx.compose.ui.tooling.preview.Preview
17
18 @Composable
19 fun NicknameEditMode() {
20     // 状态1: 是否处于编辑模式
21     var isEditing by remember { mutableStateOf(false) }
22     // 状态2: 昵称内容
23     var nickname by remember { mutableStateOf("小明同学") }
24     // 焦点请求器: 用于编辑模式时自动获取输入框焦点
25     val focusRequester = remember { FocusRequester() }
26 }
```

```

27     Row(
28         modifier = Modifier
29             .fillMaxWidth()
30             .padding(16.dp),
31         verticalAlignment = Alignment.CenterVertically,
32         horizontalArrangement = Arrangement.SpaceBetween
33     ) {
34         // 根据模式切换显示Text或输入框
35         if (isEditing) {
36             OutlinedTextField(
37                 value = nickname,
38                 onValueChange = { nickname = it },
39                 modifier = Modifier
40                     .weight(1f) // 占满剩余宽度
41                     .focusRequester(focusRequester), // 绑定焦点请求器
42                 label = { Text("昵称") }
43             )
44         } else {
45             Text(
46                 text = nickname,
47                 fontSize = 16.sp,
48                 modifier = Modifier.weight(1f)
49             )
50         }
51
52         // 切换模式的按钮
53         Button(onClick = {
54             isEditing = !isEditing
55             // 进入编辑模式时, 延迟获取焦点 (等待输入框显示)
56             if (isEditing) {
57                 focusRequester.requestFocus()
58             }
59         }) {
60             Text(if (isEditing) "保存" else "编辑")
61         }
62     }
63 }
64
65 @Preview(showBackground = true)
66 @Composable
67 fun NicknameEditPreview() {
68     NicknameEditMode()
69 }

```

关键知识点

- `FocusRequester` 用于“主动请求焦点”，让用户进入编辑模式后无需手动点击输入框，直接就能输入；
- 切换模式时，按钮文本也同步变化（编辑→保存），符合用户习惯；
- 实际项目中，点击“保存”按钮后，需添加网络请求逻辑，将修改后的昵称上传到服务器。

案例五：输入框状态恢复（页面重建不丢失内容）

场景：用户在输入框中输入了内容，但因为“屏幕旋转”“内存不足导致页面重建”，输入的内容丢失了——这是严重影响用户体验的问题，必须解决。

核心思路

Compose的 `remember` 只能在“重组”时保存状态，无法应对“页面重建”（如旋转屏幕）。此时需要用 `rememberSaveable` ——它会把状态保存到“系统的SavedStateRegistry”中，页面重建后能恢复。

完整代码实现

Code block

```
1  import androidx.compose.foundation.layout.padding
2  import androidx.compose.material3.OutlinedTextField
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.runtime.getValue
6  import androidx.compose.runtime.mutableStateOf
7  import androidx.compose.runtime.rememberSaveable
8  import androidx.compose.runtime.setValue
9  import androidx.compose.ui.Modifier
10 import androidx.compose.ui.tooling.preview.Preview
11 import androidx.compose.ui.unit.dp
12
13 @Composable
14 fun RestoreableTextField() {
15     // 关键：用rememberSaveable代替remember，实现状态恢复
16     var inputContent by rememberSaveable { mutableStateOf("") }
17
18     OutlinedTextField(
19         value = inputContent,
20         onValueChange = { inputContent = it },
21         label = { Text("输入内容（旋转屏幕不丢失）") },
22         modifier = Modifier.padding(16.dp)
23     )
24 }
25
```

```

26 // 测试提示：预览时点击右上角的“旋转”按钮，输入内容不会丢失
27 @Preview(showBackground = true)
28 @Composable
29 fun RestoreableTextFieldPreview() {
30     RestoreableTextField()
31 }

```

进阶：自定义数据类型的状态恢复

如果状态是自定义数据类（如User），直接用rememberSaveable会报错——需要给自定义类添加“序列化”支持，步骤如下：

Code block

```

1  import androidx.compose.runtime.saveable.rememberSaveable
2  import kotlinx.serialization.Serializable
3
4  // 步骤1：给自定义类添加@Serializable注解（需要导入kotlinx序列化依赖）
5  @Serializable
6  data class User(val name: String, val age: Int)
7
8  @Composable
9  fun CustomTypeRestoreable() {
10     // 步骤2：用rememberSaveable保存自定义类型状态
11     var user by rememberSaveable {
12         mutableStateOf(User(name = "小明", age = 20))
13     }
14
15     OutlinedTextField(
16         value = user.name,
17         onChange = { user = user.copy(name = it) },
18         label = { Text("用户名") },
19         modifier = Modifier.padding(16.dp)
20     )
21 }

```

添加序列化依赖（build.gradle）：

Code block

```

1 // 自定义类型序列化依赖
2 implementation "org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.0"
3 kapt "androidx.compose.compiler:compiler:1.5.3"

```

关键知识点

- `remember` vs `rememberSaveable`：前者用于“重组”时保存状态，后者用于“页面重建”时保存状态；
- 基本数据类型（String、Int、Boolean等）可直接用`rememberSaveable`，自定义类型需序列化；
- 屏幕旋转、APP从后台返回前台导致页面重建时，`rememberSaveable`都能恢复状态。

案例六：小节回顾（输入框核心知识点串联）

这节课通过5个实战案例，覆盖了输入框开发的核心场景，我们用一张表总结关键知识点和对应解决方案：

输入框开发核心知识点速查表

开发场景	核心API/组件	关键代码
输入框下方加图标	Column（垂直布局）+ 状态判断	Column { OutlinedTextField(); Row(Icon+Text) }
软键盘适配（避免被挡）	manifest配置 + Modifier.imePadding()	android:windowSoftInputMode="adjustResize"
自定义输入框背景	Box（背景容器）+ BasicTextField	Box(background) { BasicTextField() }
切换编辑/查看模式	isEditing状态 + FocusRequester	if(isEditing) {输入框} else {Text}
状态恢复（页面重建）	rememberSaveable（代替remember）	var text by rememberSaveable { mutableStateOf("") }

输入框开发“避坑指南”

1. 优先用 `OutlinedTextField` 快速开发，需要完全自定义时用 `BasicTextField`；
2. 软键盘适配必须“manifest配置+imePadding”结合，缺一不可；
3. 页面重建时的状态恢复，一定要用 `rememberSaveable`，别用`remember`；
4. 编辑模式下主动请求焦点，能极大提升用户体验，记得用 `FocusRequester`。

课后作业：动手实现“注册页面输入框组”

需求：实现一个完整的注册页面输入框组，包含3个输入框和1个注册按钮，整合本节课所有知识点：

1. 手机号输入框：① 自定义浅蓝色圆角背景；② 输入内容后，下方显示“格式正确”绿色对勾图标；③ 未输入时显示“请输入11位手机号”提示；
2. 验证码输入框：① 右侧添加“获取验证码”按钮；② 软键盘弹出时不被挡住；③ 页面旋转后保留已输入的验证码；
3. 密码输入框：① 支持切换“明文/密文”显示（用Icon按钮控制）；② 下方显示密码强度提示（弱/中/强）；
4. 注册按钮：点击后获取三个输入框的内容，打印到日志中；
5. 整体要求：输入框垂直排列，间距16dp，页面底部的注册按钮在软键盘弹出时可见。

提示：验证码的“明文/密文”切换可通过 `visualTransformation` 参数实现

（`PasswordVisualTransformation()`为密文，`VisualTransformation.None`为明文）。重点练习多个输入框的状态管理和知识点整合，遇到问题可回顾对应案例的代码。

（注：文档部分内容可能由 AI 生成）